Manual de Usuario y Documentación Técnica - Comprimir-Archivos

1. Introducción

Comprimir-Archivos es una herramienta desarrollada en **Node.js/Electron** diseñada específicamente para automatizar el procesamiento, filtrado y compresión de archivos de facturación y soportes médicos cumpliendo con reglas estrictas de negocio.

Su propósito principal es eliminar el trabajo manual de buscar directorios locales y seleccionar archivos específicos (como XML, PDF, JSON o TXT) para comprimirlos en formato ZIP. La aplicación lee una carpeta origen (convencionalmente nombrada con el prefijo `CXC`) y, dependiendo de la entidad aseguradora seleccionada, aplica un conjunto de reglas predefinidas.

Actualizaciones de Arquitectura Recientes

En la versión más reciente, la aplicación recibió actualizaciones fundamentales:

- **Gestión de Archivos de Escritura Directa (Guardado Silencioso):** Gracias a la implementación del **File System Access API**, el motor escribe los archivos `.zip` *directamente* en tu disco duro (dentro de las carpetas correspondientes) una vez le das permisos de lectura/escritura. Ya no hay ventanas emergentes intrusivas consultando dónde "guardar" cada archivo comprimido nuevo, ni retrasos por saturación del navegador.
- **Portabilidad Nativa:** El proyecto dejó atrás las pruebas en el navegador web estándar y está completamente compilado usando **Electron**, consolidándose como un **aplicativo de escritorio independiente**. Esto garantiza un ambiente seguro (sin *Node Integration* pero con acceso eficiente al sistema de archivos local) y la entrega como un archivo Ejecutable final (.exe).
- **Procesamiento de Archivos Grandes Mejorado:** Se corrigieron cuellos de botella mediante asincronicidad secuencial (`genericProcessor`), lo que evita cruces o "carreras" de memoria cuando se comprimían docenas de archivos JSON de distintas carpetas.

Una de las grandes fortalezas de esta utilidad es su **arquitectura dinámica basada en configuración**. El código fuente de la aplicación es agnóstico a las reglas específicas de cada aseguradora; estas reglas residen enteramente en un archivo maestro llamado `entidades_config.json`. Esto permite escalar el sistema y agregar nuevas entidades en el futuro sin necesidad de tocar una sola línea de código fuente. Además, el producto final se distribuye empacado como un archivo ejecutable (standalone), brindando portabilidad total.

---

2. Guía de Instalación y Ejecución (Para usuarios finales)

La aplicación está diseñada para ser completamente portátil. Esto significa que **no requiere la instalación previa de lenguajes de programación, bases de datos, ni frameworks** en el equipo de destino.

Pasos para iniciar la aplicación:

1. **Traslado del archivo ejecutable:**
    - Toma el instalador o archivo ejecutable generado (por ejemplo, `Comprimir Archivos Setup.exe` o el archivo portable).
    - Puedes copiarlo directamente mediante una memoria USB o descargarlo en el nuevo equipo.
2. **Ejecución:**

- Si es un instalador, haz doble clic sobre él y sigue el asistente estándar (Siguiente > Siguiente > Instalar). Si es una versión portable, simplemente ubícalo en la carpeta deseada (como el Escritorio o Documentos) y haz doble clic sobre el ejecutable.
- La interfaz gráfica de la aplicación se abrirá de inmediato.

3. **Uso de la herramienta:**
   - En la pantalla principal, selecciona la carpeta matriz o raíz (usualmente las nombradas como `CXC...`) que contiene los soportes médicos.
   - Selecciona la **Entidad Aseguradora** correspondiente en la lista desplegable.
   - Presiona el botón de **Comprimir**. El sistema leerá automáticamente las carpetas y generará los archivos `.zip` en las ubicaciones correctas, mostrando un registro de las acciones en pantalla.

---

3. ¿Cómo funciona la aplicación? (Lógica actual)

El motor de la aplicación escanea la estructura de la carpeta raíz seleccionada y actúa según la **Entidad Aseguradora** elegida en la interfaz. Cuando la aplicación escanea la carpeta `CXC` , aplica filtros para saber a qué carpetas entrar, qué archivos leer y qué excluir.

La lógica general determina que cada entidad tiene necesidades únicas: algunas requieren comprimir carpetas anexas llamadas `RIPS` o `CUV` , mientras que otras piden comprimir PDFs, excluyendo aquellos que fungen como la propia factura. A continuación, se resume cómo funcionan las reglas de las seis entidades base actuales:

1. **POSITIVA COMPAÑIA DE SEGUROS S.A:**
   - **Carpetas de facturas (`CTFE...`):** Comprime los archivos PDF, excluyendo el PDF propio de la factura (aquel que arranca por "CTFE").
   - **Carpetas `RIPS` > factura:** Comprime archivos `.xml` (que comienzan por "ad"), `.txt` (que inician por "ResultadosMSPS") y `.json` originados bajo el concepto "Rips". El ZIP resultante queda en la carpeta padre `RIPS` con el formato `Rips_[Factura].zip` .
2. **SALUD TOTAL EPS S S.A:**
   - **Carpetas Numéricas (`122573` ...) > Subcarpetas `NIT_Factura` :** Busca y comprime archivos `.json` , depositándolos en una subcarpeta interna llamada `COMPRIMIDO` con el nombre de `[NIT]_[Factura].zip` .
   - **Carpetas de Factura (`CTFE...`):** Busca y comprime archivos PDF excluyendo el PDF propio de la factura.
3. **COMPAÑIA DE SEGUROS BOLIVAR S.A:**
   - **Carpetas de Factura (`CTFE...`):** Comprime archivos `.pdf` , `.txt` y `.json` , omitiendo el archivo PDF principal de la factura. El archivo `.zip` toma la nomenclatura `FAC_[Factura].zip` y queda guardado junto a las fuentes originales.
4. **COMPAÑIA MUNDIAL DE SEGUROS S.A:**
   - **Carpeta de cuentas numéricas > carpetas `CUV` y `RIPS` :** Comprime el contenido total (excepto cualquier sub-ZIP) resultando en un respectivo `CUV.zip` y `RIPS.zip` .
   - **Archivos individuales en la raíz:** Toma cada archivo `.pdf` suelto y crea un ZIP separado para *cada* uno, nombrándolo idénticamente pero reemplazando su extensión por `.zip` .
5. **EPS SANITAS:**

- **Carpeta de Factura interna (`CTFE...` > `CTFE...`):** Ingresa a la subcarpeta de la factura y comprime los `.xml`, `.pdf` y `.json`, de nuevo, omitiendo el PDF que comienza por "CTFE". El archivo asume el nombre de `[Factura].zip`.

6. **LA PREVISORA S.A COMPAÑÍA DE SEGUROS:**
   - **Carpeta de cuenta numérica > carpetas `CUV` y `RIPS`:** En cada una se comprime absolutamente todo hacia su propio archivo `CUV.zip` o `RIPS.zip`.
   - **Carpeta `SOPORTE` (en la raíz `CXC`):** Se comprimen los archivos dentro de la carpeta `SOPORTE` generando un `SOPORTE.zip`.

---

4. Guía para el Administrador: Cómo agregar nuevas entidades

Esta aplicación está desacoplada: **las reglas de compresión NO están escritas en el código de la app, sino en el archivo `entidades_config.json`**. Si requieres añadir una nueva EPS o cambiar una regla existente, basta con editar este archivo (que se aloja en la sub-carpeta `public/entidades_config.json` en código fuente, o en los *"resources"* del ejecutable para aplicaciones ya compiladas).

Estructura de código: JSON y Expresiones Regulares

Nota importante: Aunque la lógica sigue un acercamiento del tipo "Búsqueda Glob" para archivos, el motor implementa nativamente **Expresiones Regulares (RegEx)** para proveer un mejor nivel de inclusión/exclusión.

Aquí te explicamos en detalle cómo está configurada una nueva entidad, con un ejemplo estructurado:

```
"miNuevaEntidad": {
  "name": "7. NOMBRE DE LA NUEVA ENTIDAD EPS",
  "tasks": [
    {
      "folderMatch": "^CTFE",
      "filesInclude": ["\\.pdf$"],
      "filesExclude": ["^CTFE.*\\.pdf$"],
      "zipName": "{folderName}.zip",
      "outputSubfolder": ""
    },
    {
      "folderMatch": "^RIPS$",
      "subTasks": [
        {
          "folderMatch": ".*",
          "filesInclude": ["^ad.*\\.xml$", ".*\\.json$"],
          "filesExclude": [],
          "zipName": "RIPS_{folderName}.zip",
          "outputSubfolder": "Resultados"
        }
      ]
    }
  ]
}
```

¿Cómo funcionan las Expresiones Regulares en esta configuración?

Para decirle al motor en qué carpetas meterse y qué archivos escoger o evitar, usamos cadenas RegEx JSON:

- **Elegir carpetas o archivos que *finalizan* con algo (Como buscar `*.pdf` en Glob):** Para buscar todos los archivos pdf, utilizas `\\.pdf$`. *Explicación:* El `\\` permite escapar el punto; el `$` indica que así debe terminar el archivo o texto.
- **Elegir elementos que *comienzan* con algo (Como buscar `Prefijo*` en Glob):** Para indicar a la aplicación que ingrese a carpetas que comiencen con "CTFE", se utiliza la expresión `^CTFE`. *Explicación:* El símbolo `^` le indica al motor que el nombre estrictamente empieza con esas letras.
- **Uso de comodines (como buscar `*` o `**` en Glob):** El equivalente a leer "cualquier cosa intermedia" es usar `.*`. Por ejemplo, `^ad.*\\.xml$` buscará archivos que *empiecen* por "ad", tengan *cualquier* texto en medio y *terminen* en .xml. Si sólo quieres un equivalente a "leer todo", en carpeta usas `.*`.
- **Excluir archivos o negar elementos (Como `!archivo.pdf`):** La regla de exclusión se maneja mediante el arreglo `"filesExclude"`. Todo patrón asignado aquí omite automáticamente los archivos, sin importar que cumplan el arreglo `filesInclude`. En el ejemplo se pide excluir todos los PDFs que empiezan por "CTFE" usando: `"^CTFE.*\\.pdf$"`.

Explicación de los Atributos JSON

1. **`name`**: Es el título exacto que aparecerá en el menú desplegable (dropdown) del usuario en la interfaz visual.
2. **`tasks`**: Un arreglo con las instrucciones globales de búsqueda a ejecutar dentro de la carpeta seleccionada por el usuario (como entrar a carpetas CTFE o RIPS).
3. **`folderMatch`**: La condición bajo la que la aplicación va a entrar a un directorio secundario.
4. **`subTasks`**: (*Opcional*) Si en vez de archivos necesitas bajar un nivel más profundo en los directorios (ej. meterse a "RIPS" y luego buscar carpetas "CTFE" allí dentro), puedes usar `subTasks`.
5. **`filesInclude` y `filesExclude`**: Estos arreglos definen las condiciones para atrapar o rechazar archivos específicos.
6. **`zipName`**: Establece el nombre del archivo comprimido saliente. Puedes usar la variable dinámica `{folderName}` para que asuma el nombre de la carpeta de origen. Para la Mundial de Seguros, se utiliza una propiedad de compresión especial llamada `"eachFileAsZip": true`, que permite usar la variable literal `{fileNameWithoutExt}`.
7. **`outputSubfolder`**: Si la dejas vacía (`""`), el ZIP se guardará donde se encontraron los archivos. Si pones un nombre (ej. `"COMPRIMIDO"`), el sistema creará esa carpeta dinámicamente y alojará ahí el archivo ZIP resultante.